Scaling the Inputs of a Neural Network

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. The problem in Keras . . . 3
   - 3.1 The data and the network . . . 3
   - 3.2 What happens . . . 4
4. Why unscaled inputs break training . . . 5
   - 4.1 The weight of a large input gets a large gradient . . . 5
   - 4.2 The shape of the loss . . . 7
5. The fix: scale the inputs . . . 9
   - 5.1 Standardization or normalization . . . 9
   - 5.2 The same network on standardized inputs . . . 10
   - 5.3 When to scale . . . 12
6. Summary . . . 13
7. Sources . . . 13
8. Key terms . . . 13

Where this fits: Pipeline map step 5 (Engineer features). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	**5 Features**	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Feature scaling; Standardization; Normalization; Gradient descent; Improving a neural network.
- **Leads to:** Image classification with a CNN (cats vs dogs).

1. Overview

Key point: When the input features live on very different scales, a network trains slowly or not at all. Bringing every feature to the same scale before training fixes it.

Suppose a network predicts whether a user buys a product from two **features** (G-772) (input variables, one column each of the data table): age, in tens, and salary, in tens of thousands. Whether the user buys is the **target** (G-1949) (the output we predict), and each user is one **observation** (G-1374) (one record, one row of the table). Fed the raw features, the network in this Note never gets past guessing. Fed the same features standardized, it reaches 94% accuracy within 20 epochs.

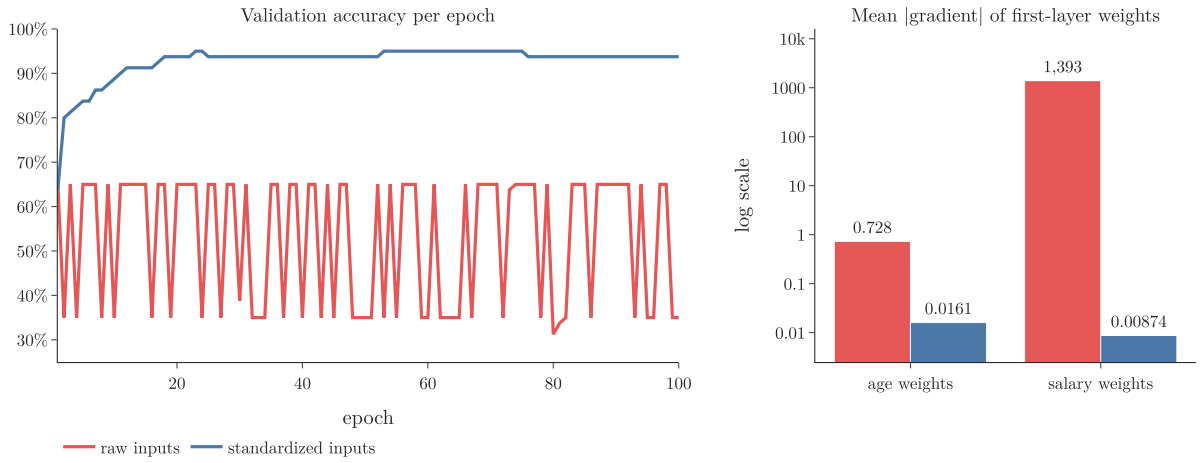


Figure 1: The same network on raw and on standardized inputs. Left: validation accuracy per epoch. Right: the average size of the gradients of the age weights and the salary weights at the start of training (log scale).

Figure 1 shows both runs and the reason. The right panel uses a **log scale**: each tick mark is ten times the one below it, so a gap of two tick marks means a factor of 100. This Note:

- shows the problem in Keras (section 3);
- explains why it happens (section 4);
- fixes it (section 5).

The Notebook (`DL-023-data-scaling-in-ann.ipynb`) runs every step.

2. Prerequisites

- **Standardization** (G-1874) and **normalization** (G-1349): the two ways to put **features** (G-772) on one scale.
- **Gradient descent** (G-862): what unscaled inputs do to the shape of the loss.
- Customer churn: a Keras network and its validation accuracy.

3. The problem in Keras

> **Key point:** On raw age and salary, the validation accuracy jumps between about 30% and 65% for 100 epochs: the network keeps predicting one class for everyone.

3.1 The data and the network

> **Key point:** 400 users, two features on very different scales, one 0/1 **target** (G-1949); one hidden layer of 128 ReLU nodes.

The data is `Social_Network_Ads.csv`, also used in the **standardization** (G-1874). It holds 400 users of a social network, and we keep three columns:

Column	Range	Meaning
Age	18 to 60	age in years
EstimatedSalary	15,000 to 150,000	estimated yearly salary
Purchased	0 or 1	**target:** 1 if the user bought the advertised product

Age is in tens and salary in tens of thousands: the salary **feature** (G-772) is about 2,000 times larger. We split 80/20, leaving 320 users for training and 80 for validation.

The network has an input layer of 2 nodes, one hidden layer of 128 ReLU nodes and one sigmoid output node, since this is binary classification. ReLU is the activation that returns its input if positive and 0 otherwise; sigmoid squeezes any number into the range 0 to 1, so the output is the probability of buying. The network is compiled with **Adam** (G-169; an optimizer, the rule that turns gradients into weight updates), the loss binary cross-entropy (the usual loss for two classes) and the metric accuracy, and trained for 100 **epochs** (full passes over the training data).

> **Python:** The network, trained on the raw columns.

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
```

```
])
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=100,
                    validation_data=(X_test, y_test))
```

3.2 What happens

> **Key point:** The accuracy never converges (settles on one value), and the loss stays in the tens.

Figure 1 (left, red) shows the validation accuracy. The accuracy jumps up and down between 31% and 65% for all 100 epochs and never settles; in the last 10 epochs it takes only two values, 35% and 65%.

The two values give the game away. Of the 80 validation users, 65% did not buy and 35% did. An accuracy of 65% means the network predicted "did not buy" for everyone; 35% means it predicted "bought" for everyone. The network flips between the two and never learns to separate the users.

The training loss tells the same story. Binary cross-entropy of a coin toss is 0.69, yet the loss starts at 3,618 and still swings between about 20 and 170 in the last 10 epochs.

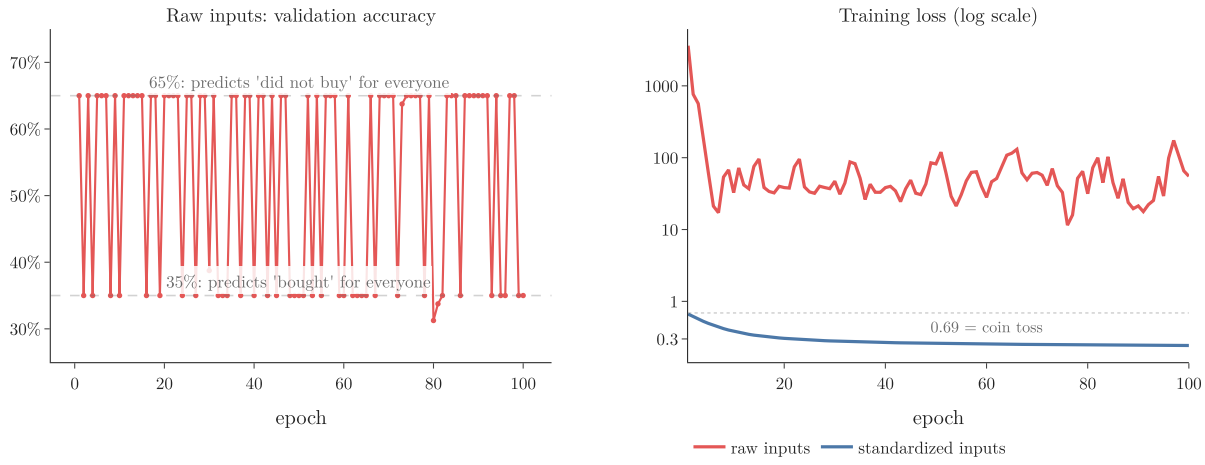


Figure 2: The raw-input run decoded. Left: validation accuracy per epoch, with the two values that mean "one class for everyone" marked. Right: training loss of both runs on a log scale; the dotted line is 0.69, the loss of a coin toss.

In Figure 2, watch how almost every point lands on one of the two dashed lines: the network is not learning slowly, it is flipping between two useless answers, while its loss never comes down to the coin-toss line that the standardized run passes in the first epoch.

4. Why unscaled inputs break training

> **Key point:** The gradient of a weight is proportional to its input (LeCun et al. 1998, §4.3). The salary weights get gradients about 2,000 times larger than the age weights, so training attends to salary and ignores age.

4.1 The weight of a large input gets a large gradient

> **Key point:** For a first-layer weight, gradient = (error signal of its node) × (its input).

Call the two inputs x_1 (age) and x_2 (salary), and their weights into one hidden node w_1 and w_2. **Backpropagation** (G-247) computes the gradient $\partial L/\partial w$ of each weight (how much the loss L changes when that weight changes). **Gradient descent** (G-862) then moves each weight against its gradient, by a step scaled by the learning rate η:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

1. **In words:** the node computes $z = w_1 x_1 + w_2 x_2 + b$. By the **chain rule** (G-371; the rule for the slope of a function built in steps), the gradient of each weight is the node's error signal $\delta = \partial L/\partial z$ times that weight's input.
2. **Formula:**
$$\frac{\partial L}{\partial w_1} = \delta \cdot x_1$$
$$\frac{\partial L}{\partial w_2} = \delta \cdot x_2$$
3. **Example:** a user aged 40 earning 80,000, and $\delta = 0.01$:
$$\frac{\partial L}{\partial w_1} = 0.01 \times 40 = 0.4$$
$$\frac{\partial L}{\partial w_2} = 0.01 \times 80{,}000 = 800$$
The salary weight gets a gradient 2,000 times larger.

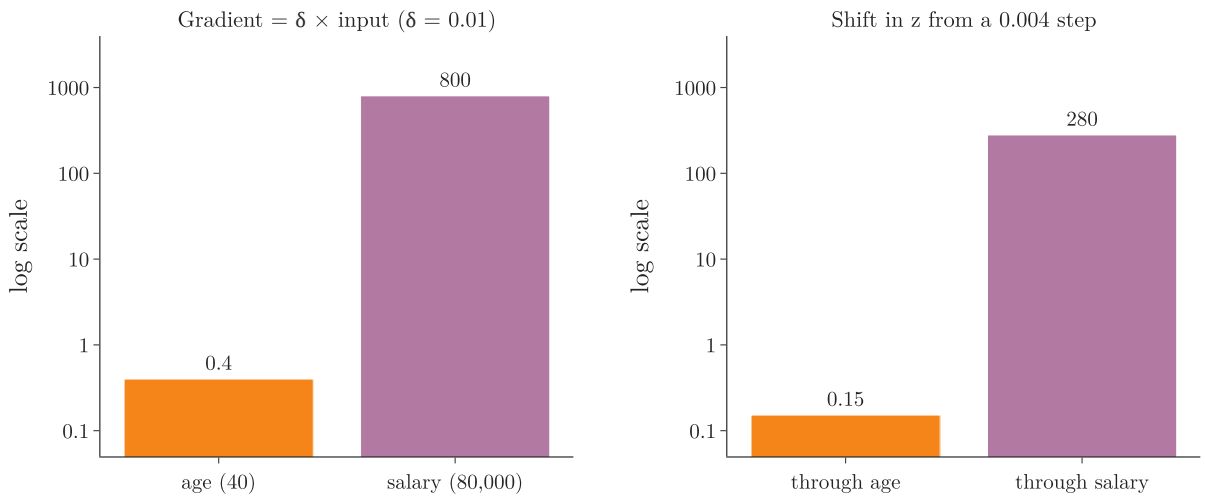


Figure 3: The worked example as bars (log scale). Left: the gradients $\delta \cdot x$ of the age and salary weights for one user. Right: how far one Adam-sized step of 0.004 moves z through a typical age (38) and a typical salary (70,000).

In Figure 3, the two panels look alike: the 2,000 times gap in the gradients (left) reappears as a similar gap (about 1,900 times) in how far z moves (right), which is why Adam's equal-sized steps do not save the raw run.

So salary takes over. With plain gradient descent, nearly all the change would go into w_2. Our network uses Adam, which moves every weight by about the same small amount (Kingma and Ba 2015, §2.1), but salary still takes over, because the same small step moves z 2,000 times further through salary than through age. Think of two volume knobs, one normal and one extremely sensitive: the same small twist barely changes the first, but makes the second jump from silent to full blast.

In numbers: in the first epoch both kinds of weight moved by about 0.004, which shifts z by about the amounts below.

$$0.004 \times 70{,}000 = 280 \quad \text{(through salary)}$$
$$0.004 \times 38 = 0.15 \quad \text{(through age)}$$

So z swings wildly from step to step, the predictions flip between all 0 and all 1, and training is unstable, as in Figure 1.

The Notebook measures this on the real network before any training (Figure 1, right). With raw inputs, the salary weights' gradients average 1,393 and the age weights' 0.73: about 1,900 times smaller. That is close to the ratio of the average salary to the average age:

$$69{,}742/38 \approx 1{,}835$$

After standardizing, both are around 0.01, within a factor of 2.

> **Extra:** The Notebook tests this by scaling one **feature** (G-772) at a time. Standardizing only salary gives 85% to 86% validation accuracy over the last 10 epochs; standardizing only age leaves the accuracy jumping between 35% and 85%. A 100 times smaller **learning rate** (G-1068) on the raw features does not help either: the network then predicts "did not buy" for everyone (65%).

4.2 The shape of the loss

> **Key point:** Unscaled inputs make the loss a long, narrow valley that **gradient descent** (G-862) zigzags across or crawls along; scaled inputs make it round.

The same effect has a geometric picture, taught in the effect of feature scaling. With inputs on very different scales, the contours of the loss are long, narrow ellipses: steep in one direction, flat in the other. A step large enough to make progress along the flat direction makes gradient descent oscillate across the valley; a step small enough for the steep direction makes it crawl along the valley floor. Either way it is slow. With inputs on the same scale, the contours are nearly circles, and gradient descent heads straight for the minimum (LeCun et al. 1998, §4.3 and §5.3).

To see this, take the simplest model, one sigmoid node, and draw its loss over the age weight and the salary weight (bias held at its best value). The loss is a surface: each pair of weights has a height. For the salary-in-thousands input and for the standardized input:

$$L(0,0) = 4.10 \quad \text{(start, salary in thousands)}$$
$$L(0,0) = 0.68 \quad \text{(start, standardized)}$$
$$L = 0.37 \quad \text{(lowest point, both)}$$

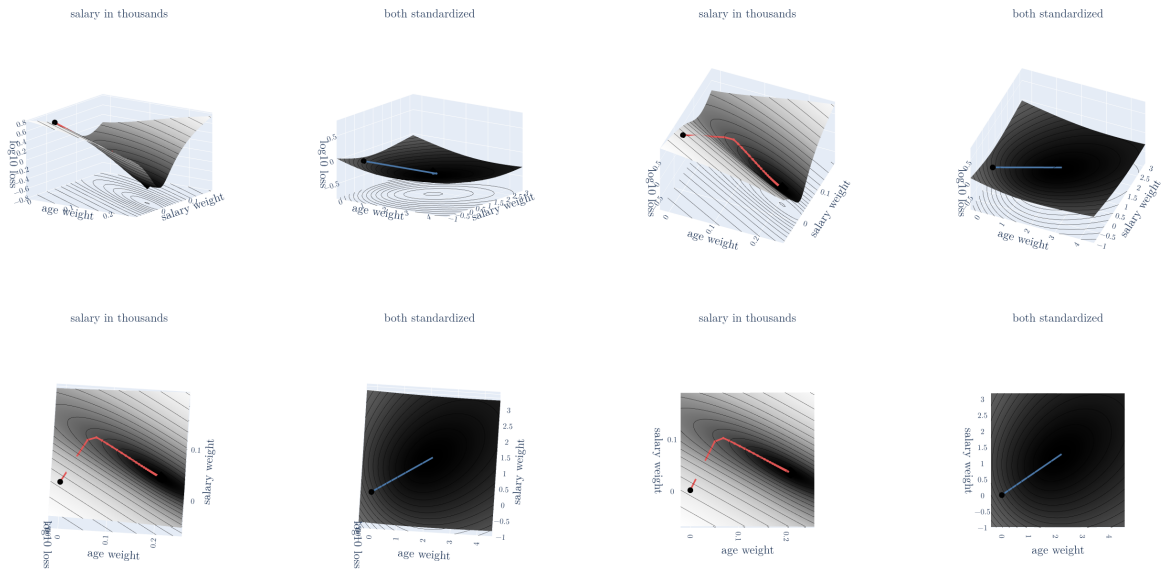


Figure 4: The loss surfaces of the sigmoid node for salary in thousands (left) and both inputs standardized (right), drawn as $\log_{10}$ of the loss. The camera tilts from a side view to the top view; the black lines join points at the same height. Coloured lines: the gradient descent paths of Figure 5; black dot: start (0, 0); black diamond: the lowest point

Figure 4 shows the two surfaces. The left one is a long, narrow valley: very steep across and almost flat along its floor. The right one is a round bowl. The contour map in Figure 5 is each surface seen from above: each line joins points at the same height; lines close together mean a steep slope, and the centre is the lowest point.

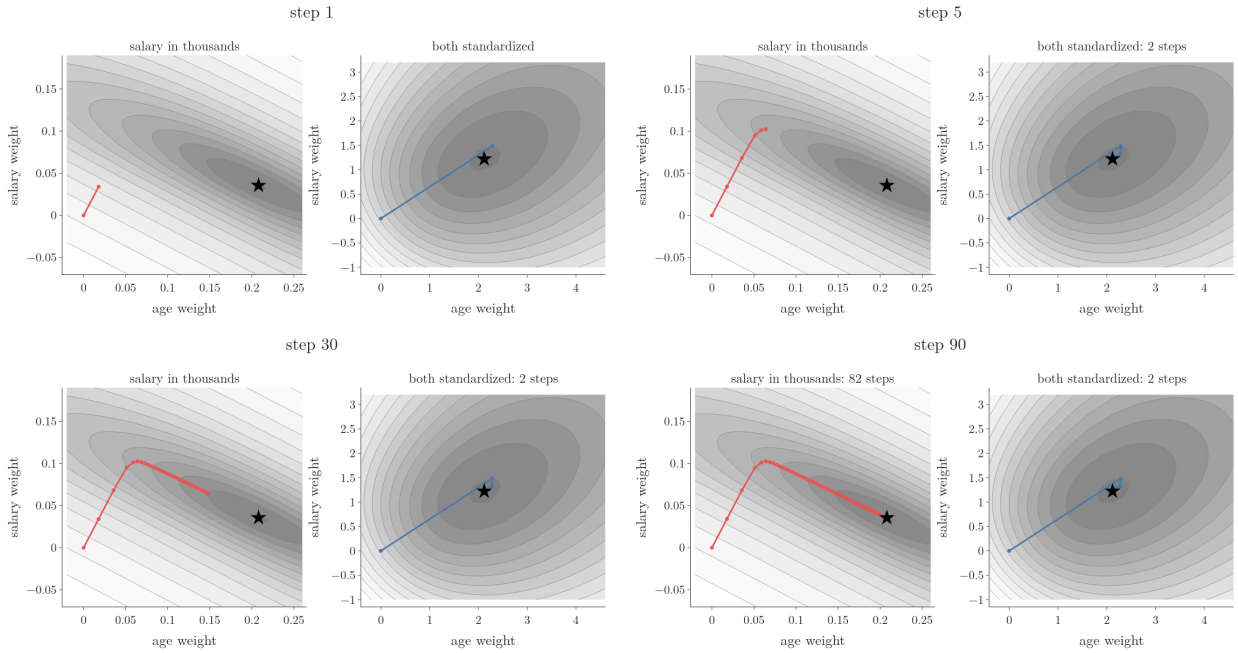


Figure 5: Gradient descent on the loss of one sigmoid node (logistic regression) for our 320 training users, drawn over the age weight and the salary weight, with the bias held at its best value. Both runs start at (0, 0) with a learning rate matched to their surface (1 divided by its steepest curvature at the minimum); the star is the minimum. Left: age in years, salary in thousands. Right: both standardized.

Figure 5 shows both pictures for our data on the simplest model, a single sigmoid node. Watch the left run: it drops quickly to the floor of the long valley, then crawls along it and needs 82 steps to come within 0.001 of the lowest loss. The standardized run on the right, on nearly round contours, gets there in 2. The narrowness can be measured as the ratio of the steepest to the flattest curvature at the minimum: about 23 on the left and 2.8 on the right. With salary in plain rupees the ratio is about 16 million, and 2,000 steps do not reach the minimum.

5. The fix: scale the inputs

> **Key point:** Standardize (or normalize) every input **feature** (G-772), fitting the scaler on the training rows only. The same network then reaches 94%.

5.1 Standardization or normalization

> **Key point:** Either works; standardization is the usual choice, normalization when a **feature** (G-772)'s minimum and maximum are known.

Both techniques put the features on one scale (see the feature scaling):

- **Standardization** (G-1874) subtracts the mean and divides by the standard deviation: every column gets mean 0 and standard deviation 1.
- **Normalization** (G-1349) (min-max scaling) subtracts the minimum and divides by the range: every column lands between 0 and 1 (see the **normalization** (G-1349)).

Which to use follows the rules for choosing a scaler. In short: normalize when the minimum and maximum are known in advance, such as a CGPA from 0 to 10; otherwise standardize, especially when the feature is close to normally distributed. Salary has no known maximum, so we standardize.

> **Extra:** Standardized values are not confined to -1 to $+1$. They have standard deviation 1, so most values fall between about -2 and $+2$, and outliers further out. Here age runs from -1.95 to 2.17 and salary from -1.61 to 2.32. Only min-max scaling guarantees a fixed range.

5.2 The same network on standardized inputs

> **Key point:** 80% validation accuracy after 2 epochs, 94% after 20, and stable from there.

> **Python:** Standardizing, then training the same network.
>
> ```
> from sklearn.preprocessing import StandardScaler
>
> scaler = StandardScaler()
> X_train_scaled = scaler.fit_transform(X_train)
> X_test_scaled = scaler.transform(X_test)
>
> history = model.fit(X_train_scaled, y_train, epochs=100,
>                     validation_data=(X_test_scaled, y_test))
> ```
>
> The scaler learns the mean and standard deviation from the training rows only, then applies them to both sets (see fit on the training set, transform both).

Why the training rows only? If the scaler also learned from the test rows, their mean and spread would leak into training, and the score on the test rows would look better than the network will do on truly new data. This mistake is called data leakage (information from the test set used while building the model) (scikit-learn User Guide, "Common pitfalls", §12.2 Data leakage).

Plotted as a scatter, the scaled data looks exactly like the raw data; only the numbers on the axes change (see the scatter plot keeps its shape). Nothing else changes either: same network, same starting weights, same 100 epochs.



Figure 6: The 320 training users before (left) and after (right) standardizing. Red crosses bought, blue dots did not.

Compare the two panels of Figure 6 point by point: the cloud has exactly the same shape, and only the axis numbers change, from 18 to 60 and 15,000 to 150,000 down to about -2 to $+2$ on both axes.

Figure 6 lets each panel choose its own axis numbers, which hides how different the raw scales are. A network does not see axis labels; it sees the numbers. Figure 7 draws the same users with one unit the same length on both axes, which is how the numbers reach the network.

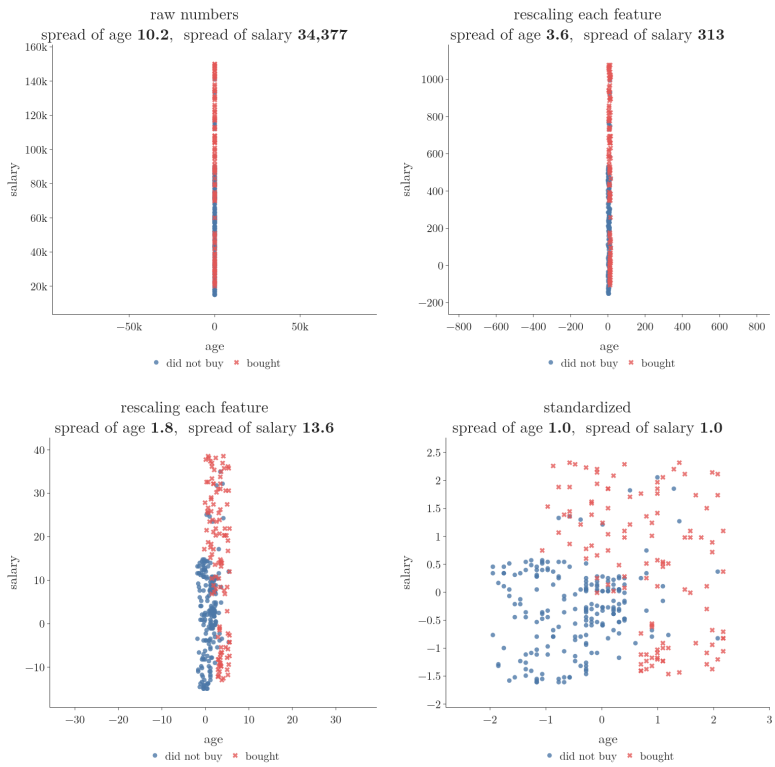


Figure 7: The 320 training users while each feature has its mean removed and is divided by its standard deviation. One unit has the same length on both axes. The title gives the standard deviation (spread) of each feature

In Figure 7, watch the shape of the cloud. In raw numbers the spread of salary is 34,377 and the spread of age is 10.2, about 3,400 times smaller, so the cloud is a thin vertical line: to the network, only salary varies. As both **features** (G-772) are standardized, the line widens into a round cloud with spread 1 in both directions, and both features vary by the same amount.

The result is Figure 1 (left, blue). The validation accuracy climbs steadily: 80% after 2 epochs, 94% after 20, and it stays there. The training loss falls smoothly from 0.67 to 0.24.

5.3 When to scale

> **Key point:** Scale the inputs of every network; **features** (G-772) already on the same known range need only a simple division to bring them to 0 to 1.

Scaling is a standard pre-processing step whenever data goes into a neural network: LeCun et al. (1998, §4.3) recommend shifting every input to mean about 0 and giving all inputs about the same spread. In cases like this one, scaling decides whether the network learns at all. Data whose features already share one range, such as the pixels of an image, which all run from 0 to 255, needs only the simplest scaling: divide by 255 (see scaling the pixels).

6. Summary

	Raw inputs	Standardized inputs
Age, salary range	18 to 60, 15,000 to 150,000	about −2 to +2 each
Mean gradient, age weights	0.73	0.016
Mean gradient, salary weights	1,393	0.009
Validation accuracy	jumps between 31% and 65%	94% from epoch 20
Training loss after 100 epochs	55	0.24

- A weight's gradient is proportional to its input, so large inputs dominate the updates.
- Geometrically, unscaled inputs stretch the loss into a narrow valley that **gradient descent** (G-862) zigzags across or crawls along.
- Standardize (or normalize) every input, so every weight gets gradients of a similar size (0.016 and 0.009 in the table).
- Fit the scaler on the training data only, so nothing from the test rows leaks into training and the test score stays a fair check.
- Make it a habit: scale before any data reaches a network.

So inputs on very different scales stop a network from training, and bringing them to one scale fixes it: 94% validation accuracy instead of guessing.

7. Sources

Built from

- CampusX, "Data Scaling in Neural Network | Feature Scaling in ANN | End to End Deep Learning Course", YouTube, https://www.youtube.com/watch?v=mzRO0cVppQ0

Other references

- LeCun, Y., Bottou, L., Orr, G. B. and Müller, K.-R. (1998). Efficient BackProp. In *Neural Networks: Tricks of the Trade*, Springer, §4.3 "Normalizing the inputs" (shift inputs to mean 0 and scale them to equal spread, so all weights learn at similar speed) and §5.3 (inputs with very different spreads make the cost surface steep in some directions and shallow in others, so learning is slow).
- scikit-learn User Guide, "Common pitfalls and recommended practices", §12.2 "Data leakage", https://scikit-learn.org/stable/common_pitfalls.html (transformations are learnt from the training data only; fitting on all the data gives overly optimistic scores).
- Kingma and Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015, §2.1 (step size about the learning rate, invariant to the scale of the gradient).

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Normalizing inputs (G-1351)	Bringing every input column of a network to the same scale before training, by standardization or normalization; without it, the weights of large-valued inputs get huge gradients and training goes badly.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.
Standardization (G-1874)	Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable.
Normalization (feature scaling) (G-1349)	The type of feature scaling that squeezes values into a fixed range, such as 0 to 1.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Adam (G-169)	An optimizer, the rule that updates a network's weights to reduce the loss: a variant of gradient descent that keeps running averages of past gradients and of their squares, giving each weight its own step size; fairly robust to its settings, so a common default.
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.